# FACTS: EBSI Simulator

Mattia Maglie - ID: 2095330

## 1. Project Overview

### 1.1 Project Description

This part of the project FACTS (Facts Authenticity and Credibility Tracking System) aims to simulate the EBSI APIs in order to make it possible to build and test applications on EBSI.

### 1.2 Technology Stack

| Component | Technology |
|---|---|
| Backend Framework | FastAPI (Python) |
| Database | PostgreSQL with SQLModel/SQLAlchemy |

### 1.3 EBSI Blockchain Simulation

The EBSI Blockchain is simulated using FastAPI and a PostgreSQL database, including all the CRUD actions.

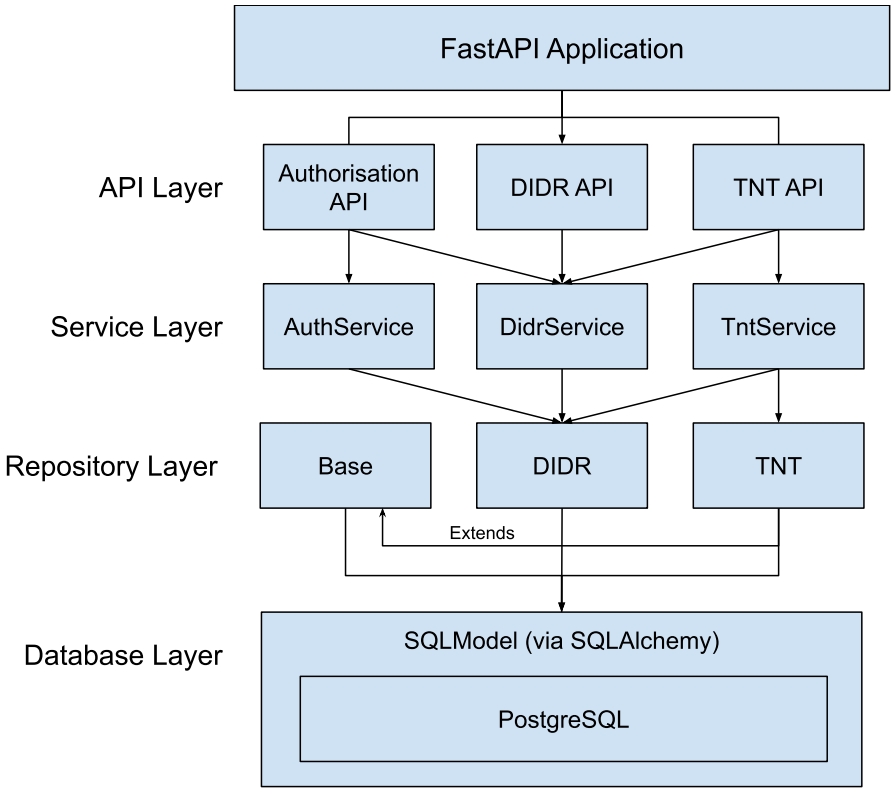
The simulation includes:

- Authorisation API (v4) - OAuth2 and OIDC implementation
- Track'n'Trace API (TNT) - Document and event tracking
- DID Registry API (v5) - Decentralized Identifier management
- Wallet Mock - Verifiable Presentation creation and ETH Transaction signature
- Issuer Mock - Verifiable Credential issuance

## 2. Architecture

The simulator is built following the "**Service - Repository**" design pattern.

### 2.1 System Architecture Diagram (excluding mock APIs)




### 2.2 Layer Descriptions

**API Layer (ebsi_sim/api/)**

- Handles HTTP requests and responses
- Implements RESTful endpoints for each service
- Handles authentication and authorization
- Input validation using Pydantic schemas

**Service Layer (ebsi_sim/services/)**

- Contains business logic
- Implements use cases and business rules
- Coordinates between API and Repository layers
- Handles transaction management

**Repository Layer (ebsi_sim/repositories/)**

- Implements Data Access Object (DAO) pattern
- Provides generic CRUD operations
- Abstracts database operations
- Returns domain models to service layer

**Model Layer (ebsi_sim/models/)**

- Defines database table schemas
- Uses SQLModel for ORM functionality
- Defines relationships between entities
- Includes database constraints

# 3. API Endpoints

## 3.1 Authorisation API (/authorisation)

| Method | Endpoint | Description |
|---|---|---|
| GET | /.well-known/openid-configuration | OpenID Provider Metadata |
| GET | /jwks | OpenID Provider's public keys |
| GET | /presentation-definitions | Presentation definition requirements |
| POST | /token | Token endpoint for OAuth2 flow |

## 3.2 DID Registry API (/did-registry)

| Method | Endpoint | Description |
|---|---|---|
| GET | /abi | Get smart contract ABI |
| GET | /identifiers | List identifiers |
| GET | /identifiers/{did} | Get a DID document |
| POST | /json-rpc | JSON-RPC endpoint |

## 3.3 Track and Trace API (/track-and-trace)

| Method | Endpoint | Description |
|---|---|---|
| GET | /abi | Get smart contract ABI |
| GET | /documents | List documents |
| GET | /documents/{document_id} | Get specific document |
| GET | /documents/{document_id}/events | List document events |
| GET | /documents/{document_id}/events/{event_id} | Get a document event |
| GET | /documents/{document_id}/accesses | List document accesses |
| GET | /accesses | List accesses |
| HEAD | /accesses | Check if DID has creator access |
| POST | /json-rpc | JSON-RPC endpoint |

## 3.4 Mock APIs

**Wallet Mock API (/wallet-mock)**

| Method | Endpoint | Description |
|---|---|---|
| GET | /create_vp | Create Verifiable Presentation from credential |
| GET | /sign_transaction | Sign Ethereum transaction |

**Issuer Mock API (/issuer-mock)**

| Method | Endpoint | Description |
|---|---|---|
| GET | /request_vc | Request Verifiable Credential |

## 4. Service Layer

### 4.1 AuthService (ebsi_sim/services/authorisation.py)

- generate_access_token() - Generate JWT access token
- generate_id_token() - Generate JWT ID token
- load_presentation() - Load presentation definition by scope
- decode_and_check_signature() - Decode and verify token signature
- find_credential() - Find and validate credential in VP
- extract_and_validate_credentials() - Extract credentials from VP
- get_verifiable_presentation() - Decode and validate VP
- check_scope_constraints() - Validate scope constraints
- create_token() - Create access and ID tokens from VP

### 4.2 DidrService (ebsi_sim/services/didr.py)

- get_did_document() - Retrieve DID document by DID
- count_did_documents() - Count DID documents with optional controller filter
- list_did_documents() - List DID documents with pagination and controller filter
- get_verification_method() - Get verification method by ID
- list_verification_methods() - List verification methods with pagination
- insert_did_document() - JSON-RPC: Create new DID with verification methods and relationships
- update_base_document() - JSON-RPC: Update DID base document
- add_controller() - JSON-RPC: Add DID controller
- revoke_controller() - JSON-RPC: Revoke DID controller
- add_verification_method() - JSON-RPC: Add verification method to Identifier
- add_verification_relationship() - JSON-RPC: Add verification relationship (authentication, assertionMethod, etc.)
- revoke_verification_method() - JSON-RPC: Revoke verification method (set expiration)
- expire_verification_method() - JSON-RPC: Set expiration date for verification method
- get_abi() - Get DID Registry contract ABI
- handle_rpc() - Handle JSON-RPC requests with scope and access validation

JSON-RPC Scope Requirements:

| **Method** | **Required Scopes** |
|---|---|
| insertDidDocument | didr_invite, didr_write |
| updateBaseDocument | didr_write |
| addService | didr_write |
| revokeService | didr_write |
| addController | didr_write |
| revokeController | didr_write |
| addVerificationMethod | didr_write |
| addVerificationRelationship | didr_write |
| revokeVerificationMethod | didr_write |
| expireVerificationMethod | didr_write |
| rollVerificationMethod | didr_write |
| sendSignedTransaction | didr_invite, didr_write |

### 4.3 TntService (ebsi_sim/services/tnt.py)

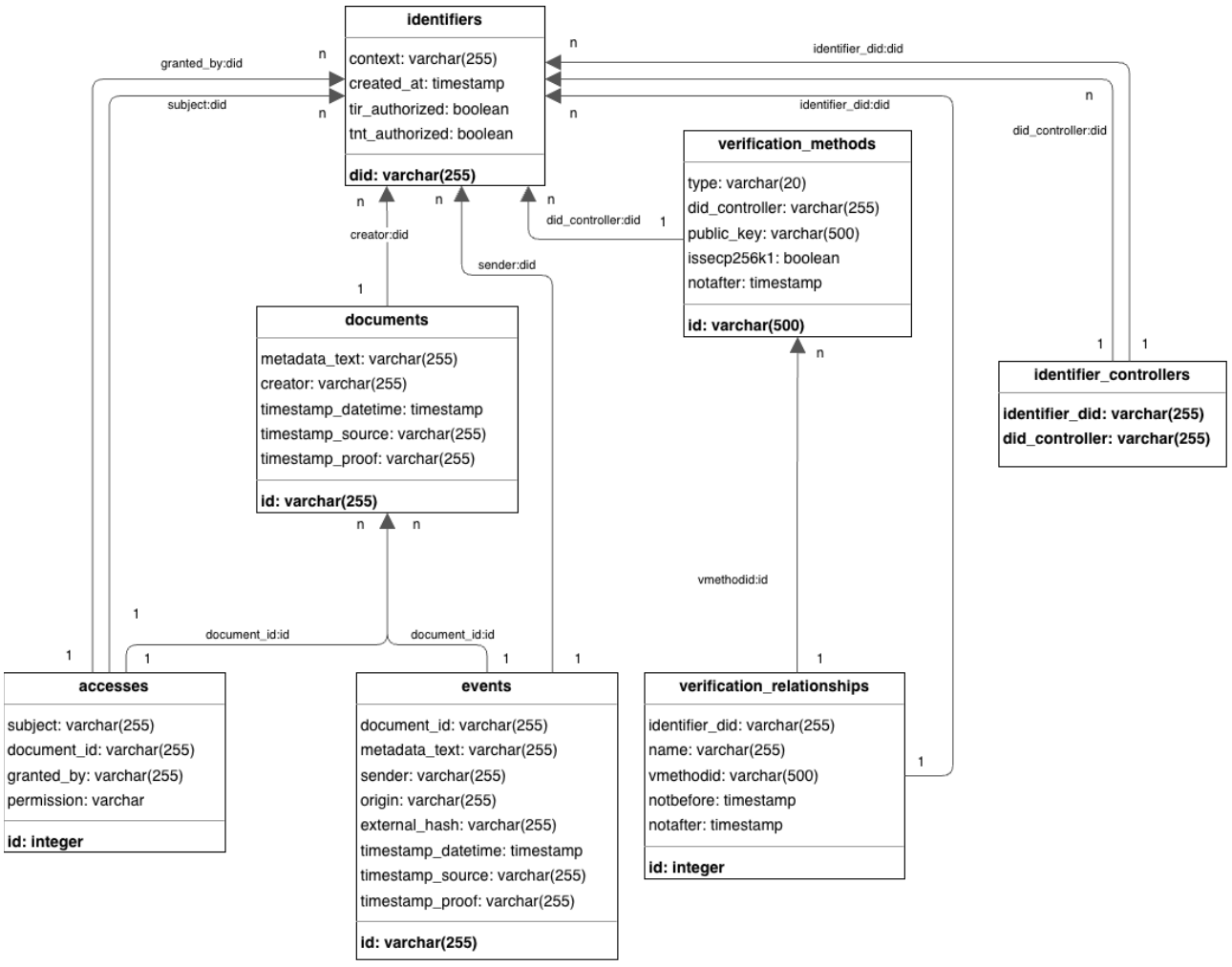
- get_document() - Retrieve document by hash
- count_documents() - Count documents with filters
- list_documents() - List documents with pagination
- count_accesses() - Count accesses with filters
- list_accesses() - List accesses with pagination
- count_events() - Count events with filters
- list_events() - List events with pagination
- authorise_did() - JSON-RPC: Authorize/unauthorize DID for TNT operations
- create_document() - JSON-RPC: Create new document with metadata and timestamps
- remove_document() - JSON-RPC: Remove document (creator only)
- grant_access() - JSON-RPC: Grant write/delegate access to document
- revoke_access() - JSON-RPC: Revoke document access
- write_event() - JSON-RPC: Write event to document with access control
- get_abi() - Get TNT contract ABI
- handle_rpc() - Handle JSON-RPC requests with scope and method constraint validation

JSON-RPC Scope Requirements:

| **Method** | **Required Scopes** |
|---|---|
| authoriseDid | tnt_authorise |
| createDocument | tnt_create |
| removeDocument | tnt_write |
| grantAccess | tnt_write |
| revokeAccess | tnt_write |
| writeEvent | tnt_write |
| sendSignedTransaction | tnt_authorise, tnt_create, tnt_write |

# 5. Database structure

## 5.1 Relational model



# 6. Exception Handling

## 6.1 Core Exceptions (ebsi_sim/core/exceptions.py)

- **EBSIError**: Base exception for all EBSI errors (Status Code: 500)
- **EBSIDatabaseError**:  Database-related errors (Status Code: 500)
- **EBSIDuplicateError**: Duplicate entry errors (Status Code: 409)
- **EBSIAuthError**: Authentication errors (Status Code: 401)
- **EBSIRequestError**: Request processing errors (Status Code: 400)
- **EBSINotFoundError**: Resource not found errors (Status Code: 404)

## 6.2 Service-Specific Exceptions

- **AuthServiceError**: Base error for Auth service operations (Status Code: 500)
- **AuthServiceAuthError**:  Authentication error in Auth service (Status Code: 401)
- **AuthServiceNotFoundError**: Resource not found in Auth service (Status Code: 404)
- **AuthServiceRequestError**: Request error in Auth service (Status Code: 400)
- **DidrServiceError**: Base error for DID Registry service (Status Code: 500)
- **DidrServiceDuplicateError**: Duplicate entry in DID Registry service (Status Code: 409)
- **DidrServiceAuthError**: Authentication error in DID Registry service (Status Code: 401)
- **DidrServiceNotFoundError**: Resource not found in DID Registry service (Status Code: 404)
- **DidrServiceRequestError**: Request error in DID Registry service (Status Code: 400)
- **TntServiceError**: Base error for TNT service (Status Code: 500)
- **TntServiceDuplicateError**: Duplicate entry in TNT service (Status Code: 409)
- **TntServiceAuthError**: Authentication error in TNT service (Status Code: 401)
- **TntServiceNotFoundError**: Resource not found in TNT service (Status Code: 404)
- **TntServiceRequestError**: Request error in TNT service (Status Code: 400)

# 9. Testing

## 9.1 Test Files

| Test File | Description |
|---|---|
| test_authorisation.py | Tests for Authorisation API endpoints |
| test_didr.py | Tests for DID Registry API endpoints |
| test_tnt.py | Tests for Track'n'Trace API endpoints |
| conftest.py | Pytest fixtures configuration (client and mock db) |

## 9.2 Test Coverage

The test suite covers:

- API endpoint response status codes
- OIDC configuration discovery
- Presentation definitions retrieval
- DID document CRUD operations
- TNT document retrieval and listing
- TNT document events retrieval and listing
- Authentication endpoints

# 10. Configuration

## 10.1 Environment Variables (ebsi_sim/core/config.py)

| Setting | Description |
|---|---|
| DB_USER | PostgreSQL username |
| DB_PASSWORD | PostgreSQL password |
| DB_HOST | PostgreSQL host |
| DB_PORT | PostgreSQL port |
| DB_NAME | PostgreSQL database name |
| AUTH_PUBLIC_KEY | Authorization service public key (ES256) |
| AUTH_PRIVATE_KEY | Authorization service private key (ES256) |
| ETH_ADDRESS | Ethereum address |
| ISSUER_AUTHENTICATION_PUBLIC_KEY | Issuer authentication public key (ES256K) |
| ISSUER_AUTHENTICATION_PRIVATE_KEY | Issuer authentication private key (ES256K) |
| ISSUER_ASSERTION_PUBLIC_KEY | Issuer assertion public key (ES256) |
| ISSUER_ASSERTION_PRIVATE_KEY | Issuer assertion private key (ES256) |
| ISSUER_DID | Issuer DID |

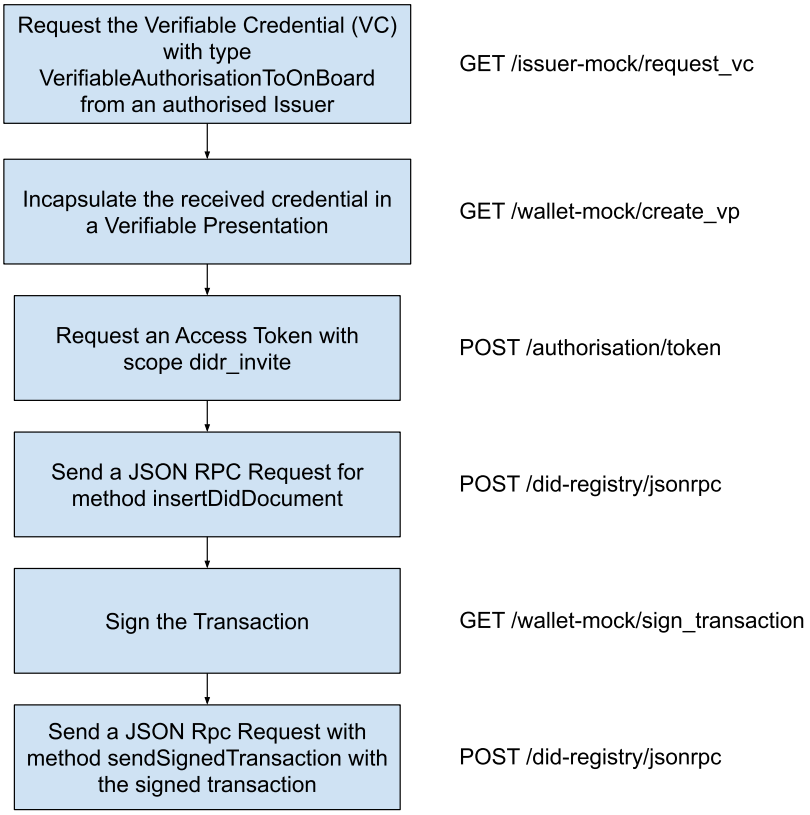
## 10.2 Docker configuration (ebsi_sim/core/config.py)

The project includes Docker configuration for easy deployment and development:

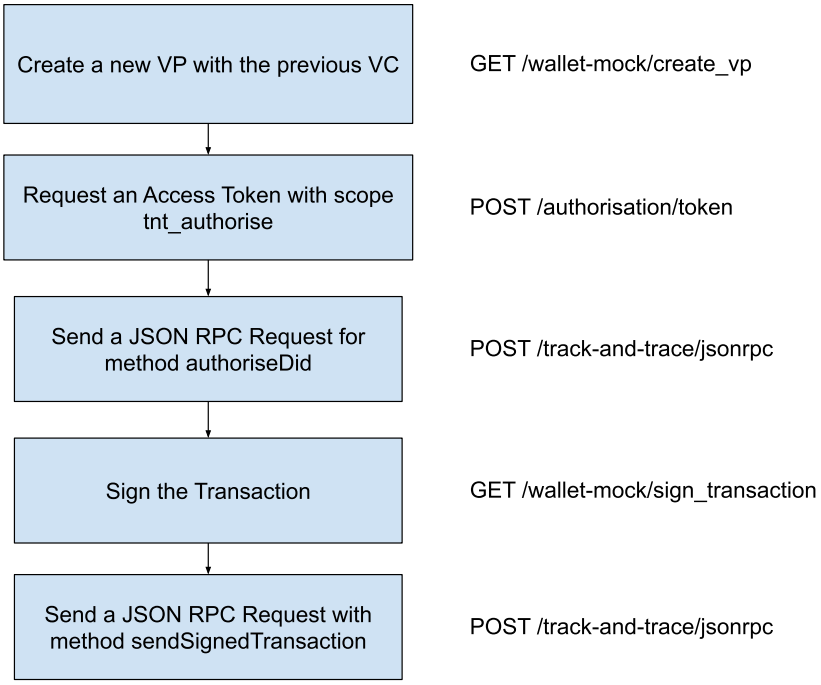
- Dockerfile - Containers definition for the EBSI Simulator
- compose.yaml - Docker Compose configuration for multi-container setup

# 11. Simulator Base Flows

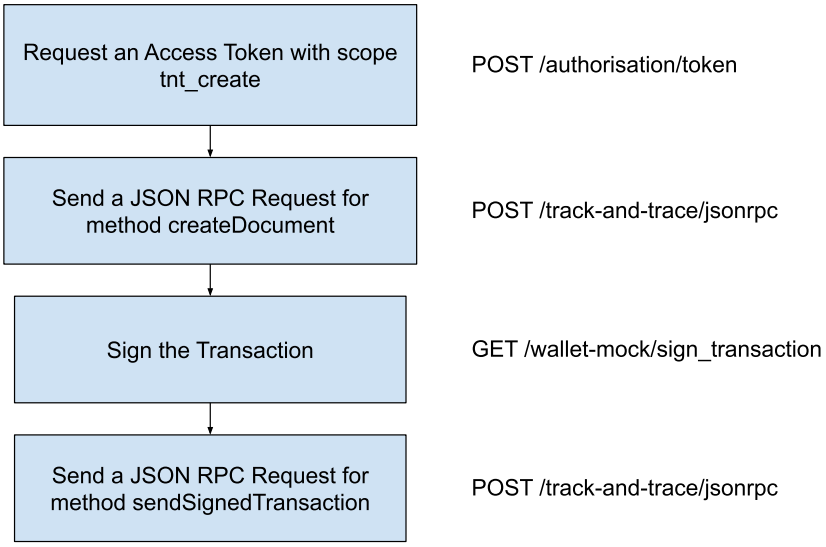
## 11.1 Create DID Document

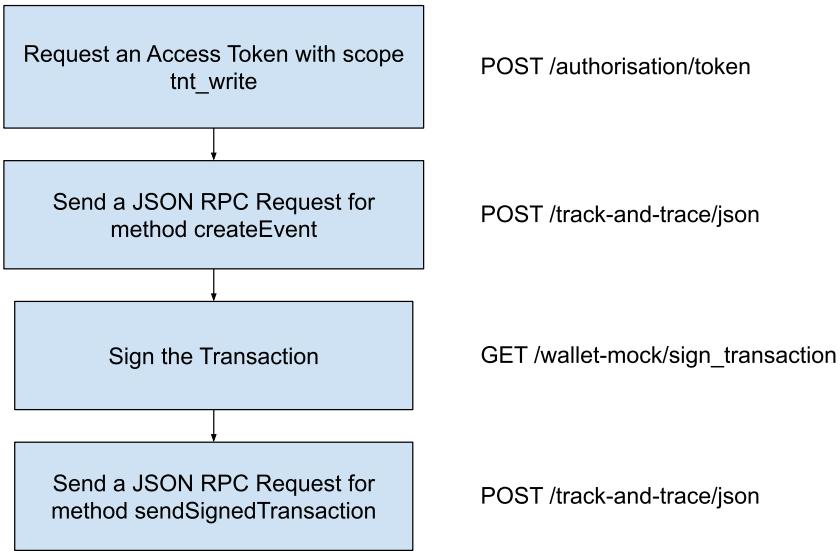



## 11.2 Get authorization to operate on TNT



## 11.3 Create TNT Document




## 11.4 Add event to a TNT Document